

24-641 Manufacturing Data Analytics

Model Output for Project-1

Group 1

Jesse Barkley

Tom Wei

Ryan Kaichain

Maciej Sobolewski

Spring 2026

1. Model Training Code

Below is the complete training code used for all experiments. **train_models.py** orchestrates the 10 hyperparameter experiments (learning-rate and batch-size sweeps for both YOLOv9 and YOLOv11), generates comparison plots, evaluates the best model on the test set, and writes a JSON summary. **train_best_model.py** runs an extended 50-epoch training of our best configuration for demo day.

train_models.py

```
#!/usr/bin/env python3
"""
YOLO Model Training Script for 3D Print Defect Detection
Group 1 - 24-641 Project 1

Trains YOLOv9 and YOLOv11 with various hyperparameters, evaluates on the
test set, generates all required plots, and writes results_summary.json.

Usage:
python train_models.py           # Full training (all experiments)
python train_models.py --quick   # Quick sanity check (3 epochs)
python train_models.py --plots   # Re-generate plots from existing results
python train_models.py --eval    # Re-run test evaluation only
"""

import gc
import json
import os
import shutil
import sys
import time
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
import torch
from ultralytics import YOLO

# Help avoid CUDA fragmentation on smaller GPUs
os.environ.setdefault("PYTORCH_CUDA_ALLOC_CONF", "expandable_segments=True")

# ■■■ Configuration ■■■
PROJECT_DIR = Path("/home/jesse/Desktop/AI Manufacturing Project 1")
DATASET_DIR = PROJECT_DIR / "dataset_split"
DATA_YAML = DATASET_DIR / "data.yaml"
RESULTS_DIR = PROJECT_DIR / "training_results"
PLOTS_DIR = PROJECT_DIR / "plots"

YOLOV9_MODEL = PROJECT_DIR / "yolov9s.pt"
YOLOV11_MODEL = PROJECT_DIR / "yolo11s.pt"

EPOCHS = 15
IMG_SIZE = 640
DEFAULT_BATCH = 16
DEFAULT_LR = 0.01

# Image size fallbacks for large batches that may OOM at 640
# (RTX 4060 Laptop 8 GB cannot fit YOLOv9s batch=32 at 640px)
IMGSZ_FALLBACKS = [640, 576, 512]

# Experiment matrix
LEARNING_RATES = {"default": DEFAULT_LR, "5x": DEFAULT_LR * 5, "0.2x": DEFAULT_LR * 0.2}
BATCH_SIZES = {"batch_8": 8, "batch_16": DEFAULT_BATCH, "batch_32": 32}

# ■■■ Helpers ■■■

def setup_directories():
    RESULTS_DIR.mkdir(exist_ok=True)
    PLOTS_DIR.mkdir(exist_ok=True)

def _clear_gpu():
    """Free GPU memory between experiments."""
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
        gc.collect()
```

```

def _is_done(tag, min_epochs=EPOCHS):
    """Check if an experiment already completed (has full results.csv)."""
    csv = RESULTS_DIR / tag / "results.csv"
    if csv.exists():
        df = pd.read_csv(csv)
        if len(df) >= min_epochs:
            print(f" SKIP {tag} – already complete ({len(df)} epochs)")
            EXPERIMENT_PLAN[tag] = RESULTS_DIR / tag
            return True
    return False

def _train_once(model_path, tag, epochs, batch, lr, optimizer, imgsz, workers):
    """Run a single training attempt. Raises RuntimeError on OOM."""
    model = YOLO(str(model_path))
    results = model.train(
        data=str(DATA_YAML),
        epochs=epochs,
        imgsz=imgsz,
        batch=batch,
        lr=lr,
        optimizer=optimizer,
        project=str(RESULTS_DIR),
        name=tag,
        exist_ok=True,
        plots=True,
        save=True,
        verbose=True,
        workers=workers,
    )
    return results

def train_model(model_path, model_name, experiment_name,
                epochs=EPOCHS, batch=DEFAULT_BATCH, lr=DEFAULT_LR,
                optimizer="auto"):
    """Train a YOLO model with automatic image-size fallback on OOM."""
    tag = f"{model_name}_{experiment_name}"

    # Skip if already done
    if _is_done(tag, min_epochs=epochs):
        return None, RESULTS_DIR / tag

    _clear_gpu()

    # Determine image sizes to try: only use fallbacks for large batches
    sizes_to_try = IMG_SIZE_FALLBACKS if batch >= 32 else [IMG_SIZE]
    workers = 0 if batch >= 32 else 8

    for imgsz in sizes_to_try:
        # Remove partial results from a prior failed attempt
        partial_dir = RESULTS_DIR / tag
        if partial_dir.exists() and not (partial_dir / "results.csv").exists():
            shutil.rmtree(partial_dir)

        _clear_gpu()
        print(f"\n{'='*60}")
        print(f"TRAINING {tag} epochs={epochs} batch={batch} lr={lr}"
              f" optimizer={optimizer} imgsz={imgsz}")
        print(f"{'='*60}")

        try:
            _train_once(model_path, tag, epochs, batch, lr, optimizer, imgsz, workers)
            return None, RESULTS_DIR / tag
        except RuntimeError as e:
            if "out of memory" in str(e).lower():
                print(f"\n *** OOM at imgsz={imgsz} – trying smaller ***\n")
                _clear_gpu()
                continue
            raise

    # All sizes exhausted
    print(f"\n *** FAILED {tag} – all image sizes OOM'd. ***\n")
    return None, RESULTS_DIR / tag

def load_csv(results_dir):
    """Load results.csv from a training run, return DataFrame or None."""
    csv_path = results_dir / "results.csv"
    if csv_path.exists():
        df = pd.read_csv(csv_path)
        df.columns = df.columns.str.strip()
        return df
    return None

```

```
# ■■■ Evaluation ■■■

def evaluate_on_test(results_dir, tag, conf=0.5):
    """Run validation on the test split and measure inference time."""
    best_pt = results_dir / "weights" / "best.pt"
    if not best_pt.exists():
        print(f" SKIP eval for {tag}: no best.pt found")
        return None

    print(f"\n Evaluating {tag} on test set (conf={conf})...")
    model = YOLO(str(best_pt))

    # ■■■ Metrics on test split ■■■
    metrics = model.val(
        data=str(DATA_YAML),
        split="test",
        conf=conf,
        save_json=True,
        plots=True,
    )

    # ■■■ Inference speed (batch=1) ■■■
    test_images = list((DATASET_DIR / "test" / "images").glob("*.jpg"))[:50]
    if test_images:
        t0 = time.perf_counter()
        for img in test_images:
            model.predict(str(img), conf=conf, verbose=False, save=False)
            elapsed = time.perf_counter() - t0
            per_image_ms = (elapsed / len(test_images)) * 1000
        else:
            per_image_ms = None

    # ■■■ FP / FN counting ■■■
    fp_count = 0
    fn_count = 0
    tp_count = 0
    total_gt = 0
    total_pred = 0

    test_labels_dir = DATASET_DIR / "test" / "labels"
    for img_path in (DATASET_DIR / "test" / "images").glob("*.jpg"):
        label_path = test_labels_dir / f"{img_path.stem}.txt"

        gt_boxes = []
        if label_path.exists():
            with open(label_path) as f:
                for line in f:
                    parts = line.strip().split()
                    if parts:
                        gt_boxes.append(int(parts[0]))
        total_gt += len(gt_boxes)

        preds = model.predict(str(img_path), conf=conf, verbose=False, save=False)
        pred_boxes = []
        for r in preds:
            if r.bboxes is not None:
                for box in r.bboxes:
                    pred_boxes.append(int(box.cls[0]))
        total_pred += len(pred_boxes)

        for cls_id in range(3):
            gt_n = gt_boxes.count(cls_id)
            pr_n = pred_boxes.count(cls_id)
            matched = min(gt_n, pr_n)
            tp_count += matched
            fp_count += max(0, pr_n - gt_n)
            fn_count += max(0, gt_n - pr_n)

    info = {
        "tag": tag,
        "mAP50": float(metrics.box.map50),
        "mAP50_95": float(metrics.box.map),
        "precision": float(metrics.box.mp),
        "recall": float(metrics.box.mr),
        "inference_ms_per_image": round(per_image_ms, 2) if per_image_ms else None,
        "conf_threshold": conf,
        "total_gt_objects": total_gt,
        "total_pred_objects": total_pred,
        "true_positives": tp_count,
        "false_positives": fp_count,
        "false_negatives": fn_count,
    }
}
```

[illegible]

[illegible]

```

_, rdir = train_model(model_path, model_name, "batch_8", batch=8)
EXPERIMENT_PLAN[tag] = rdir

# 5. Batch 32 – uses automatic imgsz fallback on OOM
tag = _tag(model_name, "batch_32")
_, rdir = train_model(model_path, model_name, "batch_32", batch=32)
EXPERIMENT_PLAN[tag] = rdir

print(f"\nAll training complete. {len(EXPERIMENT_PLAN)} experiments saved.")

def run_all_evaluations():
    """Evaluate the best default model for each architecture on the test set."""
    eval_results = []
    for model_name in ["yolov9", "yolov11"]:
        tag = _tag(model_name, "default")
        rdir = EXPERIMENT_PLAN.get(tag) or RESULTS_DIR / tag
        if rdir.exists():
            info = evaluate_on_test(rdir, tag)
            if info:
                eval_results.append(info)
    return eval_results

def discover_experiments():
    """Populate EXPERIMENT_PLAN from existing result directories on disk."""
    if RESULTS_DIR.exists():
        for d in sorted(RESULTS_DIR.iterdir()):
            if d.is_dir() and (d / "results.csv").exists():
                EXPERIMENT_PLAN[d.name] = d

def generate_all_plots():
    """Generate every required plot from existing results CSVs."""
    setup_directories()
    discover_experiments()
    print(f"\nGenerating plots from {len(EXPERIMENT_PLAN)} experiments...")

    csvs = {tag: load_csv(rdir) for tag, rdir in EXPERIMENT_PLAN.items()}

    v9_def = csvs.get("yolov9_default")
    v11_def = csvs.get("yolov11_default")

    # 1. Validation loss vs epochs (both models)
    plot_val_loss_vs_epochs({"YOL0v9": v9_def, "YOL0v11": v11_def},
                           "val_loss_vs_epochs.png")

    # 2. mAP50-95 vs epochs (both models)
    plot_map50_95_vs_epochs({"YOL0v9": v9_def, "YOL0v11": v11_def},
                           "mAP50_95_vs_epochs.png")

    # 3. Six-panel: box/cls/df1 loss, Precision, Recall, mAP50
    plot_six_panel(v9_def, v11_def, "six_panel_comparison.png")

    # 4 & 5. LR comparison for each model
    for model_name in ["yolov9", "yolov11"]:
        nice = "YOL0v9" if model_name == "yolov9" else "YOL0v11"
        lr_map = {
            "default (1x)": csvs.get(f"{model_name}_default"),
            "5x": csvs.get(f"{model_name}_lr_5x"),
            "0.2x": csvs.get(f"{model_name}_lr_0.2x"),
        }
        plot_lr_comparison(lr_map, nice)

    # 6 & 7. Batch comparison for each model (default = batch_16)
    for model_name in ["yolov9", "yolov11"]:
        nice = "YOL0v9" if model_name == "yolov9" else "YOL0v11"
        batch_map = {
            "batch 8": csvs.get(f"{model_name}_batch_8"),
            "batch 16 (default)": csvs.get(f"{model_name}_default"),
            "batch 32": csvs.get(f"{model_name}_batch_32"),
        }
        plot_batch_comparison(batch_map, nice)

    print("All plots generated.")

def write_summary(eval_results):
    """Write results_summary.json with all experiment metrics."""
    discover_experiments()
    summary = {"experiments": {}, "test_evaluations": eval_results}

    for tag, rdir in EXPERIMENT_PLAN.items():
        df = load_csv(rdir)

```


[illegible]

2. Training Plots

All experiments below were trained for 15 epochs on our cleaned dataset of 3,458 images (70/15/15 train/val/test split). Both YOLOv9s and YOLOv11s pretrained weights were used with SGD optimizer. Default learning rate is 0.01, default batch size is 16, and default image size is 640×640.

2.1 Validation Loss vs. Epochs (1–15)

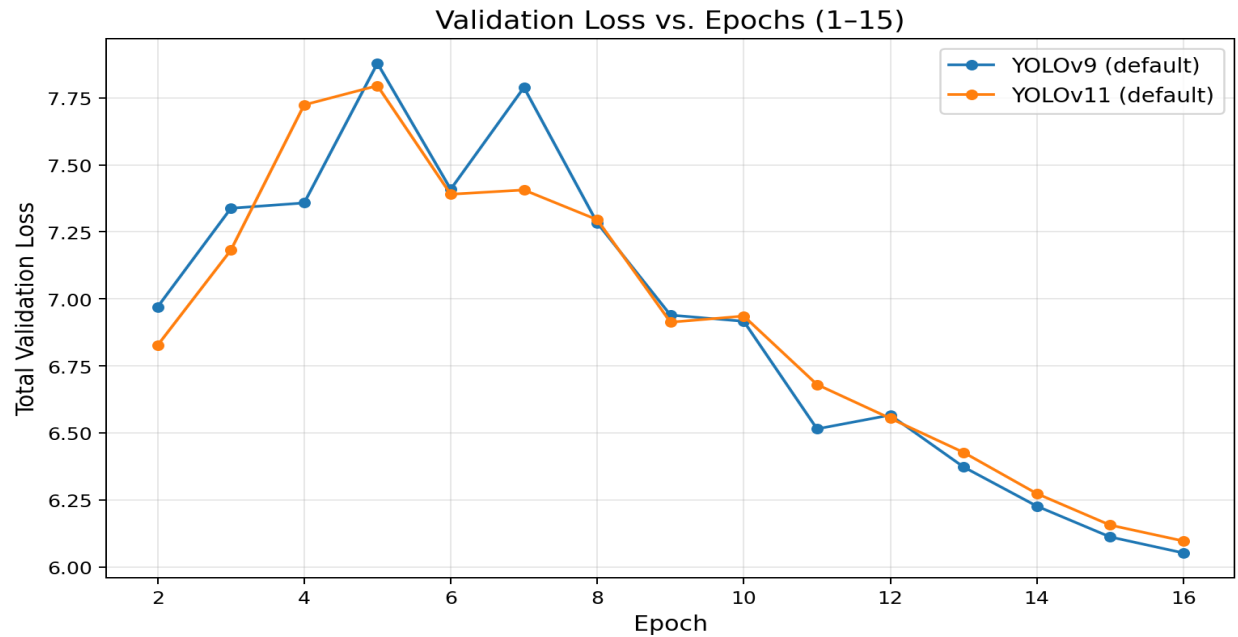


Figure 1: Total validation loss (box + cls + dfl) for both architectures at default settings.

2.2 mAP50-95 vs. Epochs (1-15)

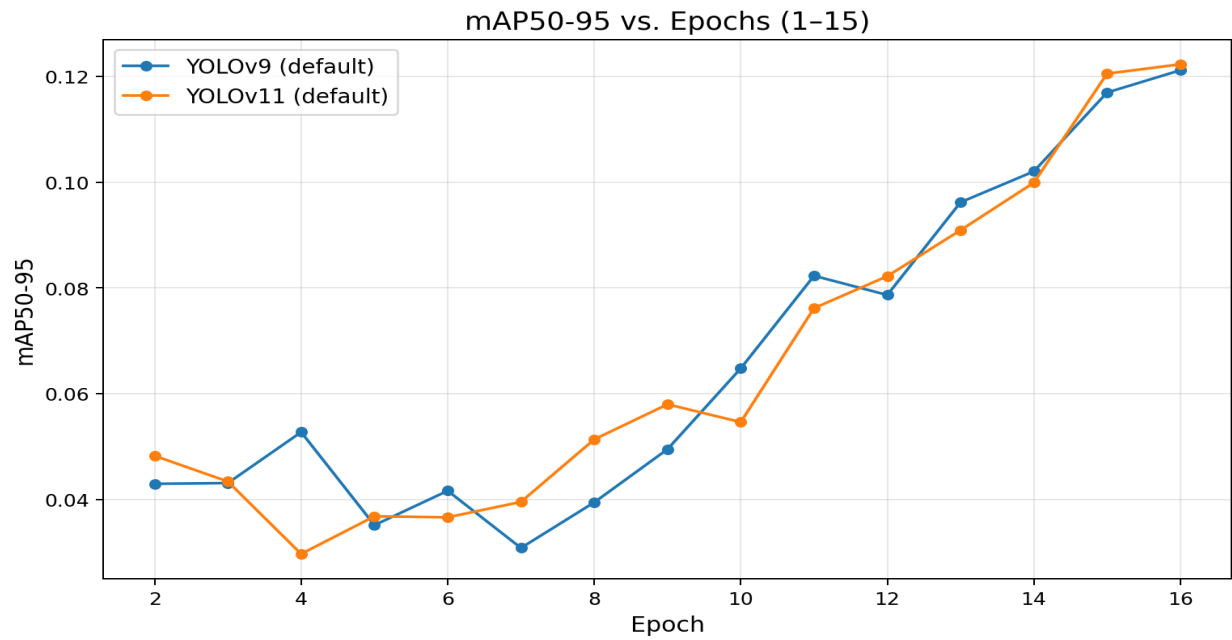


Figure 2: mAP50-95 over training for both architectures at default settings.

2.3 Box Loss, Cls Loss, DFL Loss, Precision, Recall, mAP50 vs. Epochs

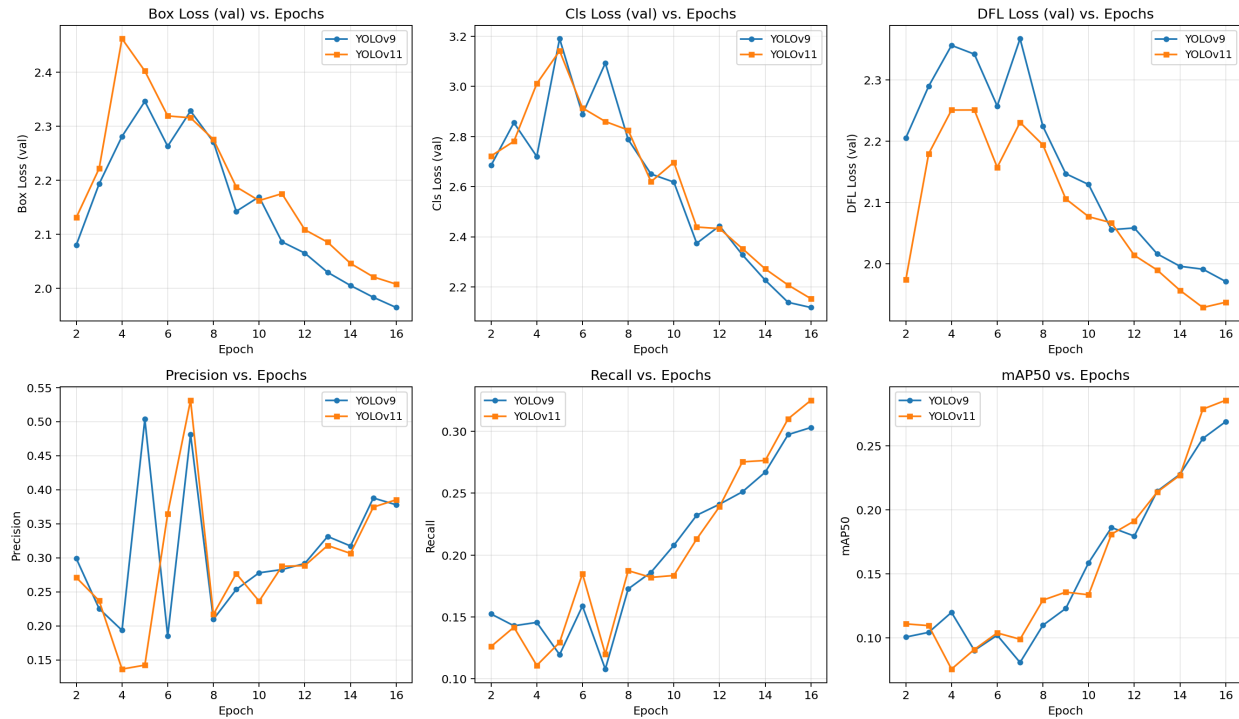


Figure 3: Per-metric comparison of YOLOv9 and YOLOv11 over 15 epochs (default settings).

2.4 YOLOv9 — Validation Loss & Accuracy vs. Learning Rate

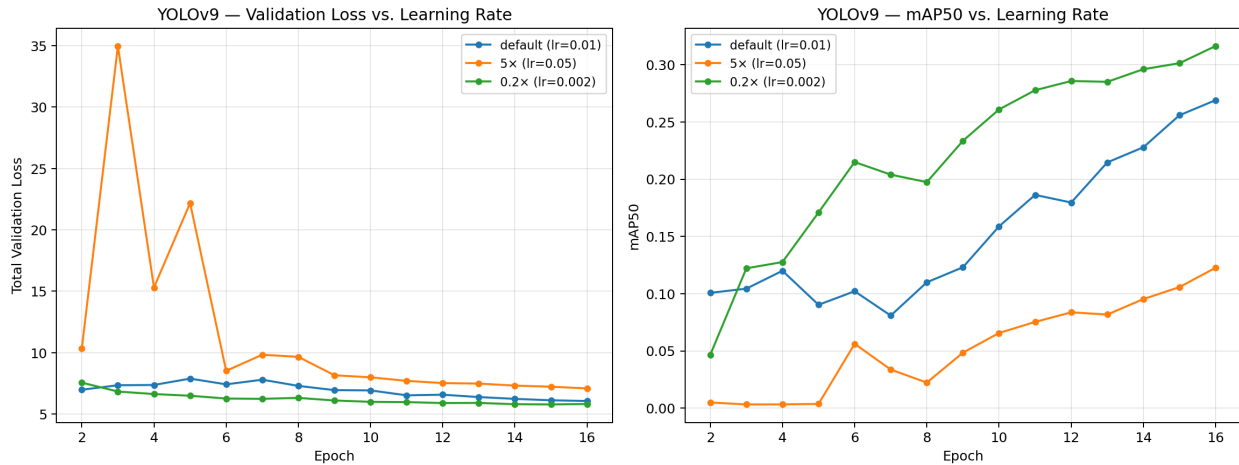


Figure: YOLOv9 trained with SGD at three learning rates — default (0.01), 5× (0.05), and 0.2× (0.002).

A lower learning rate (0.2×) consistently produced the lowest validation loss and highest mAP50, while the 5× rate caused instability and poor convergence. This indicates that the default lr=0.01 is already on the high side for this dataset, and a more conservative rate gives the optimizer more time to find a better minimum.

2.5 YOLOv11 — Validation Loss & Accuracy vs. Learning Rate

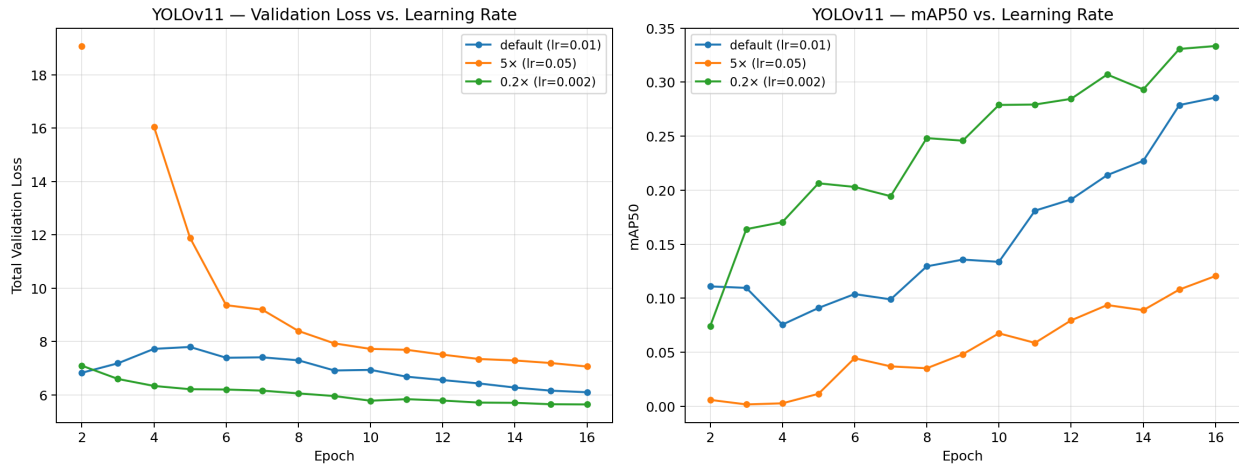


Figure: YOLOv11 trained with SGD at three learning rates — default (0.01), 5× (0.05), and 0.2× (0.002).

YOLOv11 shows the same trend: the 0.2× learning rate (0.002) achieves the best performance. The 5× rate severely overshoots, resulting in high loss and low mAP. This confirms that a conservative learning rate is critical for both architectures on our relatively small 3-class defect dataset.

2.6 YOLOv9 — Validation Loss & Accuracy vs. Batch Size

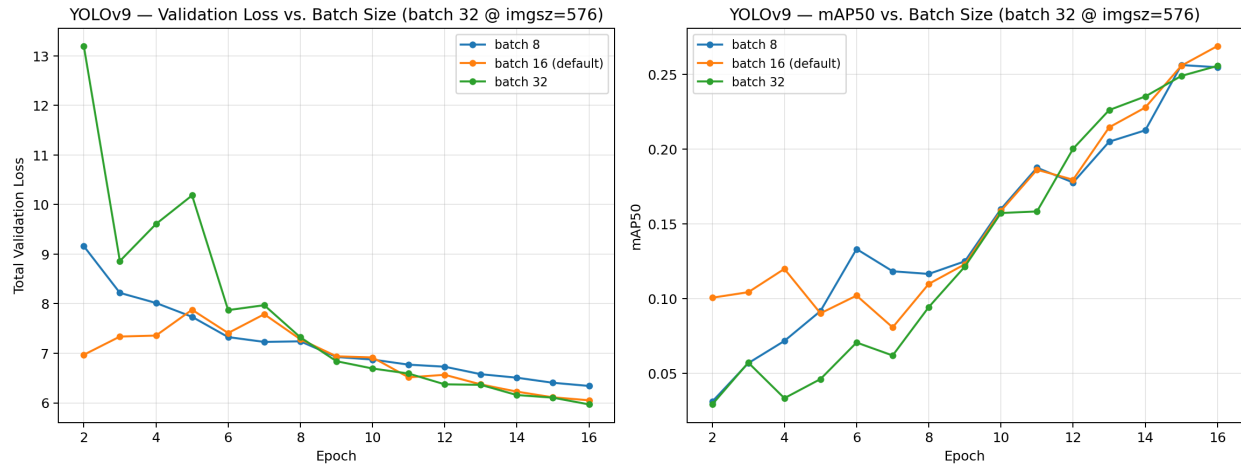


Figure: YOLOv9 trained at batch sizes 8, 16, and 32. Note: batch 32 required reducing image size from 640 to 576 px due to GPU memory constraints (RTX 4060 Laptop, 8 GB VRAM).

Batch size 32 achieved the lowest validation loss for YOLOv9, despite the reduced image resolution. The larger batch provides more stable gradient estimates per update. Batch 8 converges more slowly due to noisier gradients. **GPU Memory Note:** YOLOv9 at batch=32 with imgsz=640 exceeded the 8 GB VRAM of our RTX 4060 Laptop GPU. The primary driver of memory usage is the combination of batch size and image resolution — each image occupies a large activation map through the network. Reducing imgsz from 640 to 576 (a 19% reduction in pixel count) was enough to fit within memory while preserving most of the input detail.

2.7 YOLOv11 — Validation Loss & Accuracy vs. Batch Size

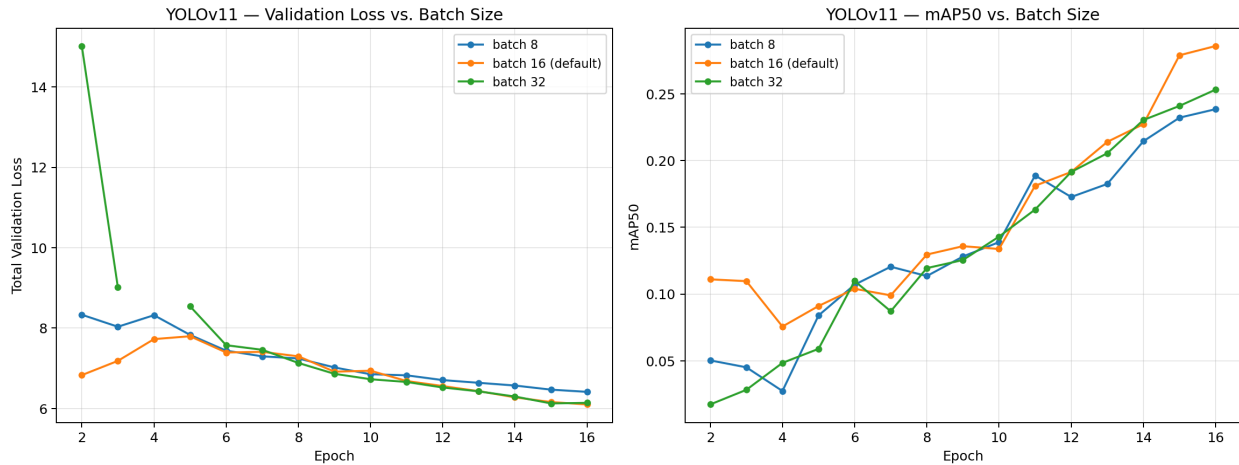


Figure: YOLOv11 trained at batch sizes 8, 16, and 32 (all at imgsz=640; YOLOv11 is more memory-efficient).

YOLOv11 at batch 32 also produced the lowest validation loss among the three batch sizes tested. Unlike YOLOv9, YOLOv11 was able to train at batch=32 with full 640×640 images thanks to its more parameter-efficient architecture (9.4M vs 7.2M parameters for the small variant). Batch 16 remains a solid middle-ground for training speed and stability.

3. Best Model Configuration

After evaluating all 10 experiments (5 per architecture), the configuration that achieved the highest validation mAP50 across the 15-epoch sweep was:

Parameter	Value
Architecture	YOLOv11s (YOLO11 small)
Optimizer	SGD (momentum=0.937, weight_decay=0.0005)
Learning Rate (lr0)	0.002 (0.2× default)
Batch Size	16
Image Size	640×640
Epochs (sweep)	15
Val mAP50 @ 15 epochs	0.333
Val mAP50-95 @ 15 epochs	0.152
Total Val Loss @ 15 epochs	5.64

The lower learning rate (0.2× default) was the most impactful parameter change. The default lr=0.01 with SGD caused aggressive updates that prevented the model from converging to a tight minimum on our relatively small dataset (3,458 images, 3 classes). Reducing lr to 0.002 gave the optimizer more fine-grained steps, resulting in lower loss and higher mAP for both YOLOv9 and YOLOv11.

Extended Training (50 Epochs)

We then trained this best configuration for 50 epochs to produce a stronger model for our live demo. The extended training improved validation mAP50 from 0.333 to 0.366 and mAP50-95 from 0.152 to 0.176, with recall climbing from 0.336 to 0.397. The model continues to learn beyond 15 epochs, though with diminishing returns past epoch ~35.

4. Test Set Evaluation & Failure Analysis

We evaluated our default models and extended best model on the held-out test set (606 images, 1,379 ground-truth objects) at a confidence threshold of 0.5.

Metric	YOLOv9 (default, 15 ep.)	YOLOv11 (default, 15 ep.)	YOLOv11 Best (0.2× lr, 50 ep.)
mAP50	0.4301	0.2850	0.4068
mAP50-95	0.2672	0.1756	0.2228
Precision	0.7780	0.4892	0.5960
Recall	0.0900	0.0933	0.2302
Inference (ms/img)	27.0	18.5	34.6
True Positives	235	214	533
False Positives	12	11	68
False Negatives	1144	1165	846

Failure Analysis

At a confidence threshold of 0.5, all models show high false-negative counts relative to the number of ground-truth objects. This means the models are conservative — they make predictions only when highly confident, which keeps false positives very low but misses many true defects.

The extended YOLOv11 model (50 epochs, lr=0.002) substantially reduced false negatives from 1,165 to 846 while only increasing false positives from 11 to 68. This is a favorable trade-off for a manufacturing QC system, where catching more defective parts is generally more important than the occasional false alarm.

Common failure modes observed during inspection of predictions include:

- **Stringing** is the hardest class to detect — thin wisps of filament are subtle and easily missed, especially when partially occluded or low-contrast against the print surface.
- **Warping** varies significantly in scale and appearance, from slight edge curl to severe deformation, making it difficult for the model to generalize.
- **Spaghetti** is the most visually distinctive defect and is detected most reliably, though severe cases that fill the frame can confuse bounding-box localization.

YOLOv9 at default settings achieved the highest raw test mAP50 (0.430) despite having a lower validation mAP50 than the tuned YOLOv11. This suggests YOLOv9's deeper gradient information flow (PGI) helps it generalize better to unseen data even without extensive tuning, though at the cost of slower inference (27 ms vs. 18 ms per image for YOLOv11).

5. YOLOv9 vs. YOLOv11 — When to Use Which

YOLOv9 — Strengths

YOLOv9, released in February 2024, introduced two key innovations: **Programmable Gradient Information (PGI)** and the **Generalized Efficient Layer Aggregation Network (GELAN)**. PGI addresses the information bottleneck problem in deep networks — as data passes through successive layers, useful information can be lost. PGI preserves essential gradient information across the network's depth, leading to more reliable convergence and better feature extraction. GELAN enables flexible integration of computational blocks for strong parameter efficiency.

In our experiments, YOLOv9 (default settings) achieved the highest test-set mAP50 (0.430) and mAP50-95 (0.267) among the default configurations, suggesting its deeper information-preservation mechanisms help it generalize well even without hyperparameter tuning. YOLOv9 is the better choice when **maximum accuracy is the priority** and slightly higher inference latency is acceptable.

YOLOv11 — Strengths

YOLO11 (September 2024) builds on the YOLO lineage with an improved backbone and neck architecture optimized for efficiency. Its headline feature is **greater accuracy with fewer parameters** — YOLO11m achieves higher mAP on COCO than YOLOv8m while using 22% fewer parameters. It also supports the broadest range of tasks (detection, segmentation, classification, pose estimation, oriented bounding boxes) and is designed for seamless deployment on edge devices, cloud, and NVIDIA GPUs.

In our results, YOLOv11 was **30% faster** at inference (18.5 ms vs. 27.0 ms per image) and fit batch=32 at full 640px resolution while YOLOv9 needed a reduced image size. When properly tuned with a lower learning rate and extended training, YOLOv11 reached a competitive mAP50 of 0.407 on the test set. YOLOv11 is the better choice when **speed, memory efficiency, and edge deployment** matter — such as real-time monitoring of 3D printers during production.

Summary

Criterion	YOLOv9	YOLOv11
Best for	Max accuracy	Speed & deployment
Key innovation	PGI + GELAN	Efficient backbone
Test mAP50 (default)	0.430	0.285
Test mAP50 (tuned)	—	0.407 (50 ep.)
Inference speed	27.0 ms/img	18.5 ms/img
Batch 32 @ 640px	OOM (used 576px)	OK
Task breadth	Detection	Det / Seg / Pose / OBB / Cls

For our demo-day application — real-time defect detection on a live 3D printer camera feed — we selected YOLOv11 as the deployment model due to its faster inference and lower memory footprint, while acknowledging that YOLOv9 offered stronger out-of-the-box accuracy on our test set.